Section 8.5: Container Orchestration Deep Dive (Kubernetes)

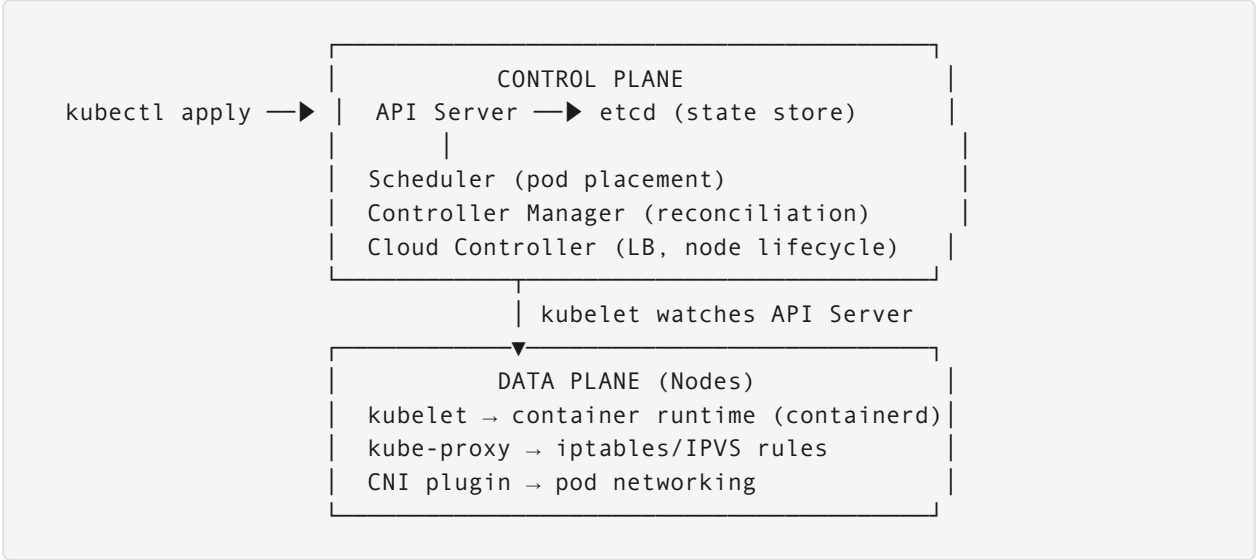
> ***Module 8 of 8 · Control Plane, Networking, Autoscaling & High Availability***

Executive Overview

Kubernetes orchestrates containerised workloads at scale through a declarative API backed by a reconciliation-loop architecture. Understanding the control plane, workload primitives, networking model, and autoscaling mechanisms is essential for production cloud-native system design. This section maps concepts to concrete design problems and interview-ready answers.

Part 1: The Kubernetes Control Plane

1.1 Component Overview and Data Flow



1.2 Control Plane Components

Component	Role	Bottleneck	Scaling Strategy
API Server	REST gateway; validates, stores in etcd	etcd write throughput	Horizontal replicas + caching layer
etcd	Cluster state store (Raft consensus)	Disk I/O; network round-trips	3 or 5 nodes for quorum; SSD disks; keep < 8GB
Scheduler	Places pods onto nodes	Scales to ~300 pods/sec typically	Multiple schedulers; scheduler extender
Controller Manager	Runs reconciliation loops	CPU on large clusters	Single leader-elected instance (fine to ~5K nodes)
Cloud Controller	Node registration, LB provisioning	Cloud API rate limits	Separate from controller manager

1.3 Reconciliation Loop Pattern

Every Kubernetes controller follows the same pattern:

```
while true:
  desired = read desired state from API server (e.g., spec.replicas = 3)
  actual  = observe actual state (e.g., running pods = 2)
  if desired != actual:
    act (create pod / delete pod / update endpoint)
  sleep(resync_period)
```

This is why Kubernetes is eventually consistent: the loop runs periodically, not instantly.

Part 2: Workload Primitives — When to Use Each

2.1 Comparison: Workload Types

Resource	Use When	Key Property	Example
Deployment	Stateless services; rolling updates needed	Arbitrary pod naming; any replica dies and is replaced	Web API, worker service

Resource	Use When	Key Property	Example
StatefulSet	Ordered startup/ shutdown; stable network identity; per-pod PVCs	Stable pod names (pod-0, pod-1); PVC survives pod deletion	MySQL, Kafka, Elasticsearch
DaemonSet	Exactly one pod per node (or node subset)	Follows node lifecycle	Log agent, Prometheus node exporter
Job	Run to completion; batch work	Tracks successful completions	Report generation, data migration
CronJob	Scheduled batch	Time-based Job creation	Nightly backups, cache warming

2.2 Deployment Rollout Strategies

```
# Rolling Update (default) — gradual replacement
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1   # Max pods down during rollout
    maxSurge: 1         # Max extra pods during rollout
# Effect: At any point, at least (replicas - 1) pods serve traffic

---
# Recreate — stop all, start all (causes downtime)
strategy:
  type: Recreate
# Use when: New version cannot coexist with old (DB schema changes, port
conflicts)
```

Blue/Green in Kubernetes (via two Deployments + Service selector swap):

```
# Green (current production)
apiVersion: apps/v1
kind: Deployment
metadata: { name: app-green }
spec:
  selector: { matchLabels: { version: green } }
---
# Blue (new version being validated)
apiVersion: apps/v1
kind: Deployment
metadata: { name: app-blue }
spec:
  selector: { matchLabels: { version: blue } }
---
```

```
# Service points to active version
apiVersion: v1
kind: Service
metadata: { name: app }
spec:
  selector: { app: myapp, version: green }  # Swap to "blue" to cut over
```

Part 3: Kubernetes Networking

3.1 Service Types — Comparison

Type	Scope	IP	Use When
ClusterIP	Cluster-internal only	Virtual stable IP	Service-to-service communication
NodePort	Cluster + external (node IP:port)	Node IP + high port	Dev/debug; not for production traffic
LoadBalancer	External via cloud LB	Public IP from cloud	Production external access; one per service
ExternalName	DNS alias to external service	No proxy	Referencing external services by stable DNS name

3.2 Ingress — L7 Routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
  annotations:
    nginx.ingress.kubernetes.io/rate-limit: "100"  # req/min per IP
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  tls:
  - hosts: [api.example.com]
    secretName: api-tls-cert
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /v1/orders
        pathType: Prefix
        backend:
```

```
          service: { name: order-service, port: { number: 80 } }
      - path: /v1/users
        pathType: Prefix
        backend:
          service: { name: user-service, port: { number: 80 } }
```

3.3 CNI Plugin Comparison

Plugin	Mechanism	Network Policy	L7 Visibility	Use When
Flannel	VXLAN overlay	No (needs Calico on top)	No	Simple clusters; getting started
Calico	BGP + iptables	Yes	No	Production; need network policies
Cilium	eBPF	Yes (L3–L7)	Yes (HTTP, Kafka, gRPC)	Advanced observability; service mesh features
Weave	VXLAN mesh	Yes	No	Multi-cloud; no BGP required

3.4 Network Policies

```
# Allow only api-service pods to reach database on port 5432
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-isolation
  namespace: production
spec:
  podSelector:
    matchLabels: { role: database }
  policyTypes: [Ingress, Egress]
  ingress:
  - from:
    - podSelector: { matchLabels: { role: api } }
    ports:
    - { protocol: TCP, port: 5432 }
  egress: []  # Database cannot initiate outbound connections
```

Part 4: Autoscaling

4.1 HPA — Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: order-api-hpa }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-api
  minReplicas: 3
  maxReplicas: 50
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization, averageUtilization: 70 }
  - type: External   # Custom metric: SQS queue depth
    external:
      metric: { name: sqs_queue_depth, selector: { matchLabels: { queue:
orders } } }
      target: { type: AverageValue, averageValue: "100" }
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60    # Don't scale up for 60s after last scale
    scaleDown:
      stabilizationWindowSeconds: 300   # Wait 5 min before scaling down
```

4.2 Three-Layer Autoscaling

Layer	Component	Triggers	Timescale
Pod	HPA	CPU, memory, custom metrics	Seconds to minutes
Pod resource	VPA	Historical CPU/memory usage	Minutes (requires pod restart)
Node	Cluster Autoscaler	Pending pods / underutilised nodes	Minutes

```
Load spike → HPA adds pods → Pods pending (no node capacity)
                           → Cluster Autoscaler provisions new node
                           → Pods scheduled to new node (~2–5 min total)
```

```
Traffic drops → HPA removes pods → Nodes underutilised
                                 → Cluster Autoscaler drains and removes node
```

Part 5: Interview Design Problems

Problem 1 — High Availability for Stateless Applications

Question: How does Kubernetes achieve HA for stateless apps? What happens when a node fails?

Solution:

HA Setup:

```
# Anti-affinity: spread pods across nodes and AZs
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions: [{ key: app, operator: In, values: [order-api] }]
        topologyKey: topology.kubernetes.io/zone  # One pod per AZ
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: kubernetes.io/hostname
    whenUnsatisfiable: DoNotSchedule
    labelSelector: { matchLabels: { app: order-api } }
```

Node failure timeline:

```
T+0s:    Node stops responding
T+40s:   Node controller marks node as NotReady (node-monitor-grace-period)
T+40s:   Endpoints controller removes pod IPs from Service endpoints
         (Traffic stops routing to dead pods immediately)
T+40s:   Pods on failed node marked for eviction (pod-eviction-timeout)
T+45s:   Scheduler places replacement pods on healthy nodes
T+60s:   New pods pass readiness probe; traffic resumes
T+300s:  (Default) Pods marked Terminating if node unrecovered

Net user impact: ~40–60s reduced capacity, no hard downtime if 3+ replicas
```

Required for zero downtime: 1. Minimum 3 replicas spread across 3 AZs 2. Readiness probes configured (traffic only to ready pods) 3. PodDisruptionBudget: `minAvailable: 2` to prevent all pods evicted simultaneously

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: order-api-pdb }
spec:
  minAvailable: 2
  selector: { matchLabels: { app: order-api } }
```

Problem 2 — Stateful Database on Kubernetes

Question: Should you run a production PostgreSQL database on Kubernetes? If so, how do you ensure data survives node failures?

Solution:

```
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: postgres }
spec:
  serviceName: postgres
  replicas: 3  # Primary + 2 standbys
  selector: { matchLabels: { app: postgres } }
  template:
    spec:
      containers:
      - name: postgres
        image: postgres:16
        volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
        env:
        - name: POSTGRES_REPLICATION_MODE
          value: master  # or slave
  volumeClaimTemplates:
  - metadata: { name: data }
    spec:
      accessModes: [ReadWriteOnce]
      storageClassName: gp3-encrypted  # Cloud SSD with encryption
      resources: { requests: { storage: 100Gi } }
```

Node failure with StatefulSet:

```
Problem: Pod stuck in "Terminating" when node goes offline (Kubernetes cannot
         confirm pod stopped before rescheduling — could cause split-brain)

Safe process:
1. Force delete pod:   kubectl delete pod postgres-0 --force --grace-period=0
2. Kubernetes reschedules postgres-0 to healthy node
3. PVC reattaches (cloud volumes support cross-AZ reattach in same region)
```

```
4. Postgres starts with data intact

Volume reattach time: ~30-90s (cloud-specific)
```

Trade-off: Kubernetes DB vs managed RDS:

Concern	K8s StatefulSet	Managed RDS
Data durability	EBS/PD with snapshots	Built-in multi-AZ replication
Failover time	2–5 min manual or operator	30–60s automatic
Backup	Manually configure	Automated with point-in-time recovery
Ops overhead	High (you manage replication)	Low
Cost	Lower infra cost	Higher managed service premium
Control	Full (extensions, config)	Limited by provider

> ***Recommendation:*** *Use managed RDS/CloudSQL unless you need full control (custom extensions, specific PG version, cost at scale). Use Kubernetes StatefulSet for stateful services that aren't databases (e.g., Kafka, Elasticsearch) where managed services are prohibitively expensive.*

Problem 3 — Autoscaling Under Traffic Spikes

Question: Your service normally handles 100 RPS but expects a 10× spike during a flash sale. How do you design autoscaling to handle this without cold-start delays?

Solution:

```
# Pre-scale before the spike: CronJob to increase minReplicas
apiVersion: batch/v1
kind: CronJob
metadata: { name: pre-scale-flash-sale }
spec:
  schedule: "0 9 * * 5"  # 9 AM every Friday (flash sale time)
  jobTemplate:
    spec:
      template:
        spec:
```

```
          serviceAccountName: hpa-patcher
          containers:
          - name: scaler
            image: bitnami/kubectl
            command:
            - kubectl
            - patch
            - hpa/order-api-hpa
            - --patch
            - '{"spec":{"minReplicas":20}}'
```

Scaling math:

```
Normal load:    100 RPS × 200ms avg = 20 concurrent → 5 pods at 70% CPU
Flash sale:   1,000 RPS × 200ms avg = 200 concurrent → 50 pods needed

HPA reaction time: ~2–3 min (metric sample interval + stabilization)
Pod startup time:  ~30–60s (pull + init + readiness probe)
Total: 3–4 min to full capacity

With pre-scaling (minReplicas=20 before spike):
  Already at 20 pods when traffic hits
  HPA scales remaining 30 during the event (2–3 min)
  Users see < 1s increased latency in first minute only
```

```
# Aggressive HPA scale-up for known spike patterns
behavior:
  scaleUp:
    stabilizationWindowSeconds: 0   # Scale immediately on spike
    policies:
    - type: Percent
      value: 100           # Double pods every 15 seconds
      periodSeconds: 15
  scaleDown:
    stabilizationWindowSeconds: 600  # Don't scale down for 10 min (avoid
thrashing)
```

KEDA (Kubernetes Event-Driven Autoscaler) for queue-based scaling:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: order-processor }
spec:
  scaleTargetRef: { name: order-processor }
  minReplicaCount: 0      # Scale to zero when queue empty (cost saving)
  maxReplicaCount: 100
  triggers:
  - type: aws-sqs-queue
    metadata:
```

```
      queueURL: https://sqs.us-east-1.amazonaws.com/123/order-queue
      queueLength: "50"   # One pod per 50 messages
```

Quick Reference

Concept	Value / Rule
etcd recommended nodes	3 or 5 (odd number for quorum)
etcd performance limit	~8GB data; SSD required
Node failure detection time	40s (node-monitor-grace-period default)
HPA metric evaluation interval	15s
HPA scale-down stabilization window	300s default
Cluster Autoscaler node provision time	2–5 min (cloud-dependent)
StatefulSet pod naming	`<name>-0`, `<name>-1`, … (stable)
PodDisruptionBudget purpose	Guarantee minimum available pods during voluntary disruption